Computer Architecture (Practical Class)
Assembly: Bit-level Operations

Luís Nogueira Nuno Pereira Raquel Faria

Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

{lmn,nap,arf}@isep.ipp.pt

2018/2019

Bit-level Operations

Bit-level operations modify one or more bits at a time and are used to manipulate binary numbers (stored in variables or registers)

- Are very efficient operations, directly supported by the processor
- Can be used to:
  - Extract information from groups of bits
  - Perform multiplications and divisions by powers of two several times faster
  - Apply an operation to multiple data within a single variable (if the variables has groups of bits representing different data)

Two major groups of bit-level operations

- Boolean logic
  - Each bit is compared individually using the logic function specified
  - Logic functions: AND, OR, XOR, NOT
- Bit movements
  - Shift bits left/right (multiply/divide by powers of two)
  - Rotate bits left or right

Boolean Logic Operations: AND

`AND` (the " & " operator in C)

X	Y	AND
0	0	0
0	1	0
1	0	0
1	1	1

- Usage: `AND` *origin*, *destination*
  - operation: *destination* = *destination* AND *origin* (the result is placed in *destination*)
  - *origin* can be a memory address, a constant value or a register
  - *destination* can be a memory address or a register
  - the `AND` instruction can operate on numbers of 8(b), 16(w) or 32(l) bits

Boolean Logic Operations: OR

`OR` (the " | " operator in C)

X	Y	OR
0	0	0
0	1	1
1	0	1
1	1	1

- Usage: `OR` *origin*, *destination*
  - operation: *destination* = *destination* OR *origin* (the result is placed in *destination*)
  - *origin* can be a memory address, a constant value or a register
  - *destination* can be a memory address or a register
  - the `OR` instruction can operate on numbers of 8(b), 16(w) or 32(l) bits

Boolean Logic Operations: XOR

`XOR` (the " ^ " operator in C)

X	Y	XOR
0	0	0
0	1	1
1	0	1
1	1	0

- Usage: `XOR` *origin*, *destination*
  - operation: *destination* = *destination* XOR *origin* (the result is placed in *destination*)
  - *origin* can be a memory address, a constant value or a register
  - *destination* can be a memory address or a register
  - the `XOR` instruction can operate on numbers of 8(b), 16(w) or 32(l) bits

Boolean Logic Operations: NOT

`NOT` (the " ~ " operator in C)

X	NOT
0	1
1	0

- Usage: `NOT` *destination*
  - operation: *destination* = NOT *destination* (the result is placed in *destination*)
  - *destination* can be a memory address or a register
  - the `NOT` instruction can operate on numbers of 8(b), 16(w) or 32(l) bits

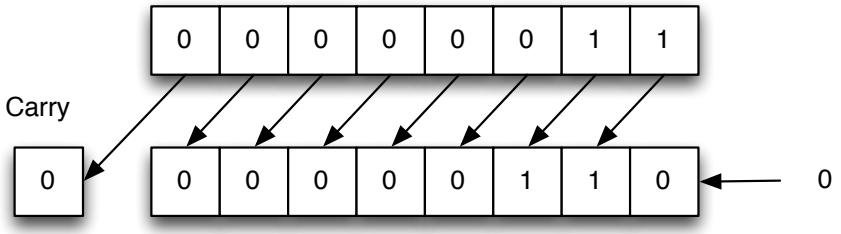
Important notes

- The Assembly instruction `NEG` changes the sign of an integer
- The logic C operator "`!`" is not a bit-level operator; it considers the entire number

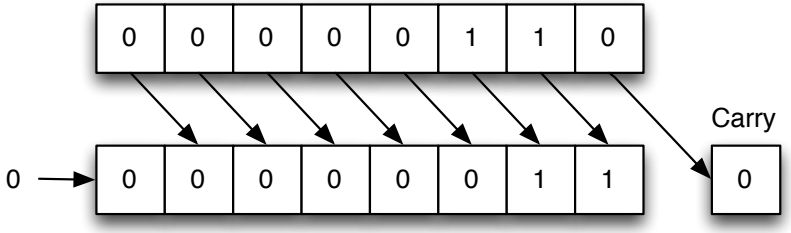
y	NOT y	NEG y	!y
0	-1	0	1
-1	0	1	0
1	-2	-1	0

Bit movement operations

- Shifting bits left and right is a very easy operation to implement on a processor. It is often used as a fast way to implement multiplication/division
- Consider a binary number system (base 2):
  - Shifting a digit left (entering a zero) corresponds to a multiplication by 2



  - Sifting a digit right (losing the rightmost digit) corresponds to a division by 2

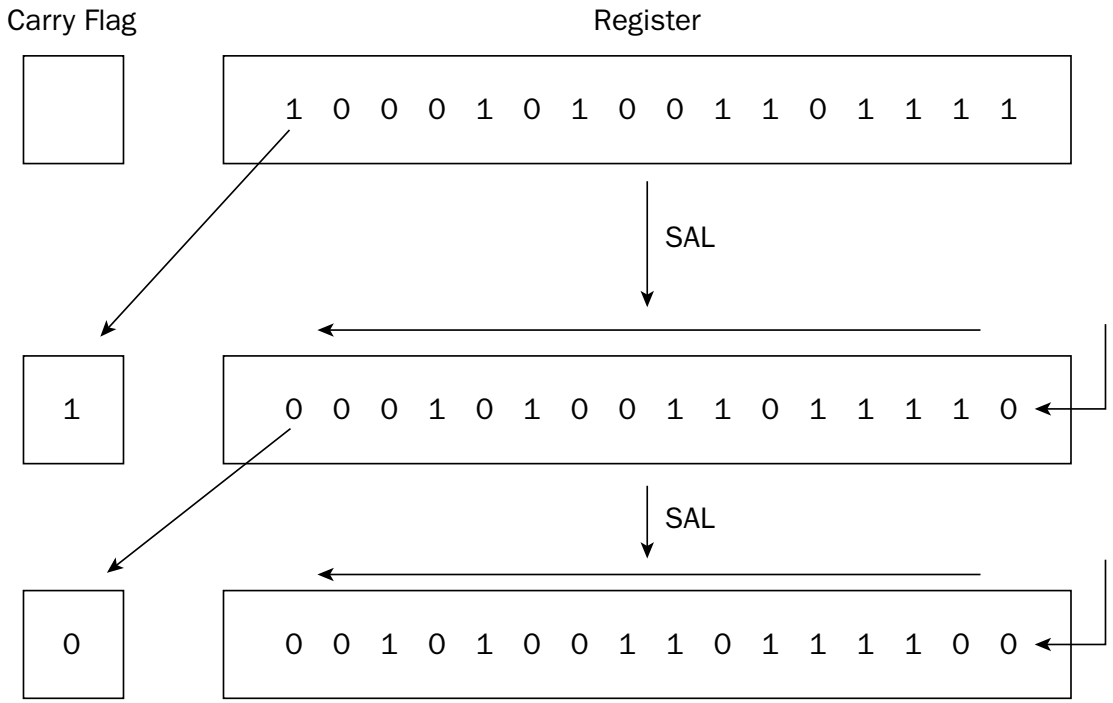


- This would be true for any base b (shifting a digit left/right corresponds to multiply/divide by b)

Shifting bits left (1/2)

`SHL/SAL` (the " << " operator in C)

- SHL and SAL are equivalent operations
  - They exist for consistency with the right shift; we will see why in a moment
- Shifts bits to the left; zeros enter on the right and the last bit to exit left goes to the carry flag



Shifting bits left (2/2)

`SHL/SAL` - Three formats

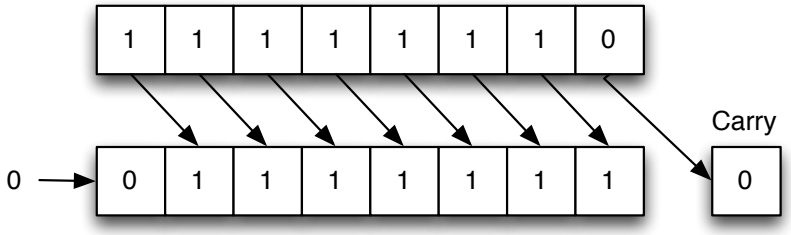
- `SHL` *destination* (or `SAL` *destination*)
  - shifts the *destination* value left one position (equivalent to *destination* $* = 2$)
- `SHL` %cl, *destination* (or `SAL` %cl, *destination*)
  - shifts the *destination* value left by the number of times specified in the CL register (equivalent to *destination* $* = 2^{CL}$)
- `SHL` $n, *destination* (or `SAL` $n, *destination*)
  - shifts the *destination* value left by the number of times specified by a constant value n (equivalent to *destination* $* = 2^n$)

- In all formats, *destination* can be a memory address or a register
- The `SHL/SAL` instructions can operate on numbers of 8(b), 16(w) or 32(l) bits

Shifting bits right (1/5)

Notes about signed numbers

- Performing a right shift on a signed integer value may adversely affect the sign of the integer
- When shifted to the right, a negative number will lose its sign if we zero-fill the leading bits

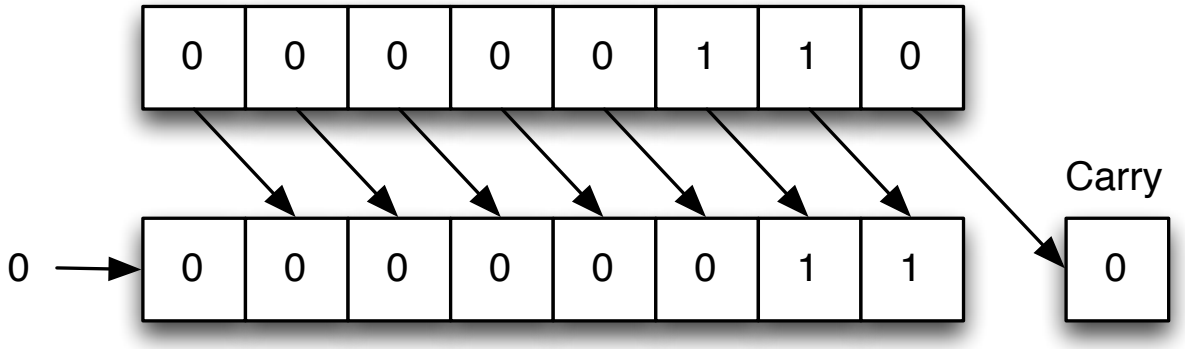


- To solve this problem there is a distinction between the right-shift instructions:
  - SHR - the **logic shift**t to the right *does not preserve the signal*;
  - SAR - the **arithmetic shift** to the right *preserves the signal*.

Shifting bits right (2/5)

SHR (the " >> " operator in C)

- Logic shift to the right; Therefore, *does not preserve the signal* of the number
- Shifts bits to the right; zeros enter on the left and the last bit to exit to the right goes to the carry flag (similar to the left shift)



Shifting bits right (3/5)

SHR - Three formats

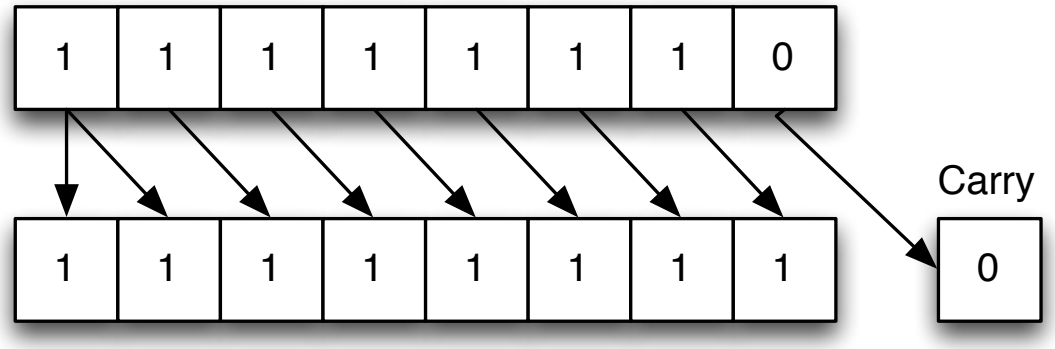
- SHR *destination*
  - shifts the *destination* value right one position (equivalent to *destination* $/ = 2$).
- SHR %cl, *destination*
  - shifts the *destination* value right by the number of times specified in the CL register (equivalent to *destination* $/ = 2^{CL}$).
- SHR $n, *destination*
  - shifts the *destination* value right by the number of times specified by a constant value n (equivalent to *destination* $/ = 2^n$)

- In all formats, *destination* can be a memory address or a register
- The SHR instruction can operate on numbers of 8(b), 16(w) or 32(l) bits

Shifting bits right (4/5)

`SAR` (the " >> " operator in C, when applied to signed integers)

- Arithmetic shift to the right; Therefore, *preserves the signal* of the number
- Shifts bits to the right; either clears or sets the bits entered on the left, according to the sign of the integer. The last bit that exits to the right goes to the carry flag



Shifting bits right (5/5)

SAR - Three formats (similar to SHR):

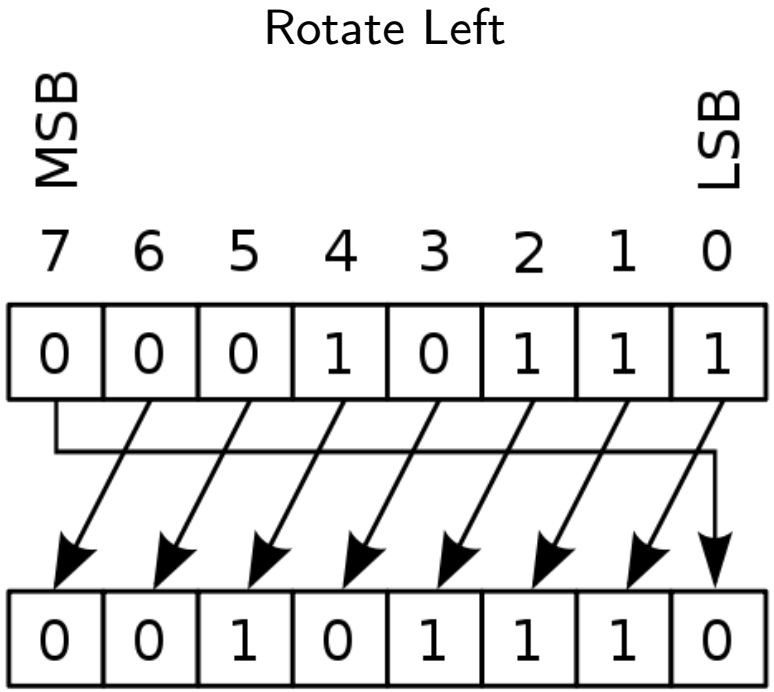
- `SAR` *destination*
- `SAR` %cl, *destination*
- `SAR` $n, *destination*

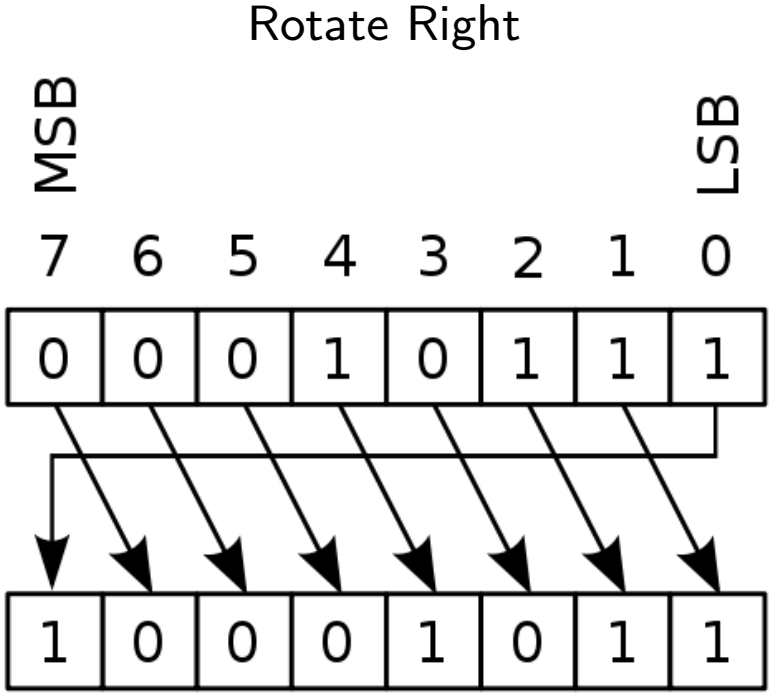
- In all formats, *destination* can be a memory address or a register
- The `SAR` instruction can operate on numbers of 8(b), 16(w) or 32(l) bits

Rotating bits

`ROL/ROR` (no equivalent operation in C)

- ROL - Bit rotation to the left
- ROR - Bit rotation to the right
- Perform just like the shift instructions, except the overflow bits are pushed back into the other end of the value instead of being dropped.





Rotating bits

ROL/ROR - Three formats (similar to shift instructions):

- `RO{L/R}` *destination*
- `RO{L/R}` %cl, *destination*
- `RO{L/R}` $n, *destination*

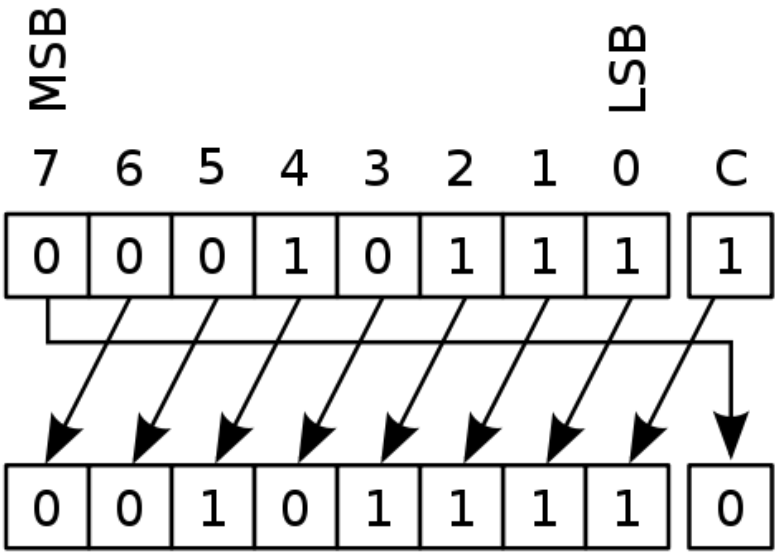
- In all formats, *destination* can be a memory address or a register
- The `ROL/ROR` instructions can operate on numbers of 8(b), 16(w) or 32(l) bits

Rotate with Carry (1/2)

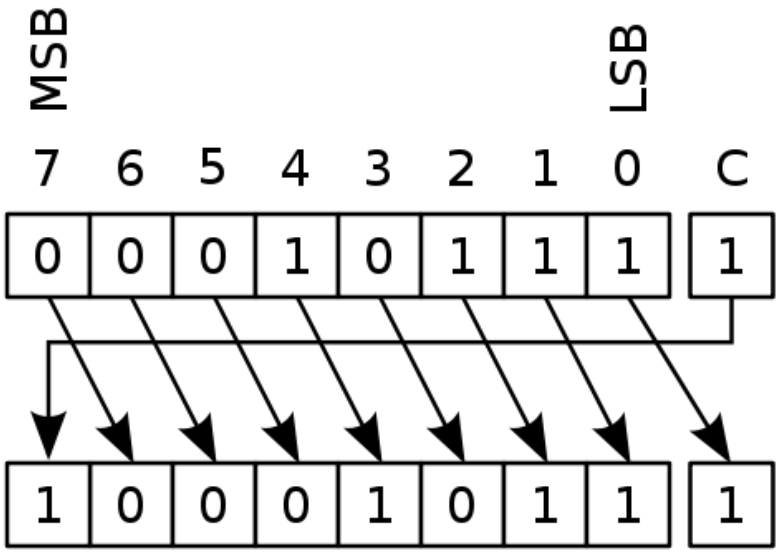
`RCL/RCR` (no equivalent operation in C)

- RCL - Bit rotation to the left *with carry*
- RCR - Bit rotation to the right *with carry*
- Perform bit rotation, but the first entering bit comes from carry and overflow bits go to carry.

Rotate Left with Carry



Rotate Right with Carry



Rotate with Carry (2/2)

RCL/RCR - Three formats (similar to ROL/ROR):

- `RC{L/R}` *destination*
- `RC{L/R}` %cl, *destination*
- `RC{L/R}` $n, *destination*

- In all formats, *destination* can be a memory address or a register
- The `RCL/RCR` instructions can operate on numbers of 8(b), 16(w) or 32(l) bits

Bit Masks (1/4)

- One common use of bit-level operations is to implement *masking* operations
- A *mask* is a bit pattern that indicates a selected set of bits within a number

- For example, assume that `%al` has a binary value of `00101100`
- How can we get the value of the 4 least significant bits?

To solve the problem we need to:

1. Determine the logic operator to use
2. Determine the mask, based on the selected operator

```
movb $0b00101100, %al
movb $0b00001111, %ah
andb %ah, %al          # %al = 0b00001100
```

Bit Masks (2/4)

- Assume now we want to replace the 4 least significant bits of %al for the 4 least significant bits of %cl.
- We need to perform two steps:
  1. Set the 4 least significant bits of %al to zero using the `and` operation
  2. Replace the bits in %al with the bits in %cl using the `or` operation

```
movb $0b00101100, %al
movb $0b00000011, %cl

movb $0b11110000, %ah
andb %ah, %al          # %al = 0b00100000
orb  %cl, %al          # %al = 0b00100011
```

Bit Masks (3/4)

- The `xor` operation can be used to set a number to zero or compare two numbers
  - The result of a `xor` operation between two equal numbers is zero

```
xorl %ebx, %ecx
jz is_equal
```

- The `xor` operation is also used to invert the bits of a register

```
mov  $0b00101100, %al
mov  $0b00001111, %ah
xor  %ah, %al          # %al = 0b00100011
```

Bit Masks - Summary (4/4)

- The **AND** operator can be used to:
  - Set bits to zero - use a bit mask with those bits set to zero
  - Get the value of a few bits - use a bit mask with those bits set to one
  - "Round" a number to a power of 2 (2^x) - use a bit mask with x least significant bits to zero
  - Get the remainder of the division by a power of two (2^x) - use a bit mask with x least significant bits to one
- The **OR** operator can be used to:
  - Set bits to one - use a bit mask with those bits set to one
  - Join the bits of two numbers - perform an **or** between the two numbers
- The **XOR** operator can be used to:
  - Check if two numbers are equal - if a **xor** between the two numbers is zero
  - Invert bits - use a bit mask with those bits set to one

Bit Masking in C - Active Learning Activity

```
#define mask(n)          ((1<<(n))-1)
#define mask2(n1,n2)     (mask(n2-n1+1)<<(n1))

#include <stdio.h>
int main ()
{
  printf("%d\n", mask(2));
  printf("%d\n", mask(3));
  printf("%d\n", mask2(1,2));
  printf("%d\n", mask2(1,3));
  return 0;
}
```

- The above code prints...
  - A. the numbers 2, 3, 2, 3
  - B. the numbers 4, 8, 6, 9
  - C. the numbers 3, 7, 6, 14
  - D. None of the above.

Sign Extension (1/3)

- Sometimes we need to convert a value to a larger data type.

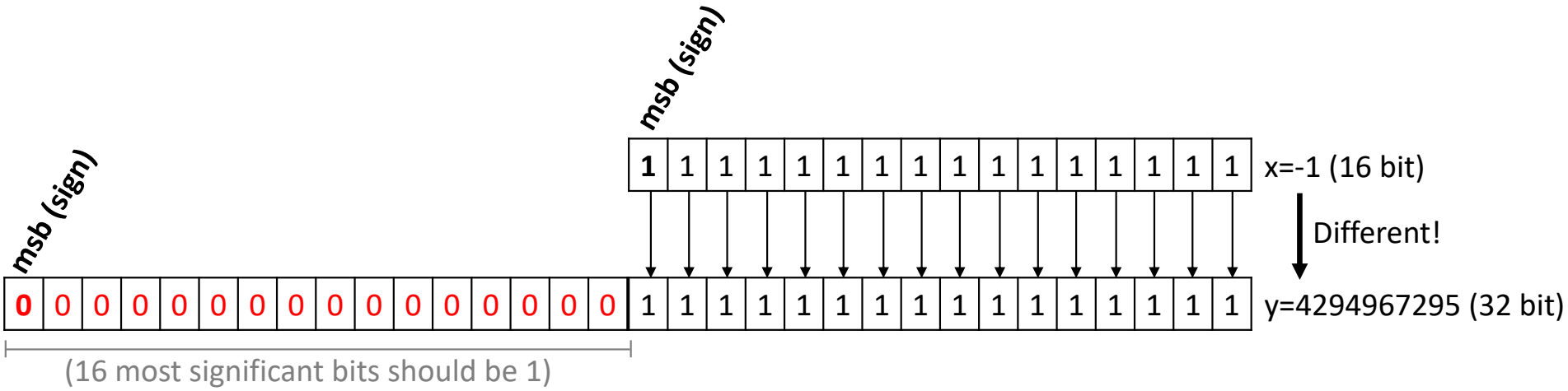
C Code to move a 16 bit value to 32 bit variable

```
short int x=100; // 16 bits (signed)
int y=0;         // 32 bits (signed)
y=(int)x;        // move 16 to 32 bit
```

Incorrect way to move a value from 16 to 32 bits

```
movl $0, %eax   # init register
movw x, %ax     # 16 bits to register
movl %eax, y    # 32 bits to variable
```

- **The assembly code on the right does not work always**. Why ?
  - What if `x` contains a negative value ?



When converting values to a larger data type...

We need to copy the sign value throughout the upper bits. This is called **sign extension** !

Sign Extension (2/3)

We could manually copy the sign value... Fortunately there is an assembly instruction to do this: `movsX` (move with sign extend).

`movsX` origin, *destination*

- *Origin* can be a memory address or a register (8 or 16 bit)
- *Destination* can be a register of 16(w) or 32(l) bits

```
movb 8(%ebp), %al
movsbw %al, %ax
movsbl %al, %ecx
```

Sign Extension (3/3)

The `cbw` (convert byte to word), `cwd` (convert word to double word) and `cdq` (convert double word to quad word) can be used to produce a correct dividend before a division instruction

- `cbw` - converts the signed byte in AL to a signed word in AX by copying the sign bit of AL
- `cwd` - converts the signed word in AX to a signed double word in EAX by copying the sign bit of AX
- `cdq` - converts the signed double word in EAX to a signed quad word in EDX:EAX by copying the sign bit of EAX to all bits of EDX

```
movl 8(%ebp), %eax
cdq
movl 12(%ebp), %ecx
idivl %ecx
```

Practice: Sum bytes

- Implement, in Assembly, a function `void sum_bytes(int a, int b, char *sum)`
  - Sums the most significant byte of the integer `a` with the least significant byte of `b`
  - Writes the result in the memory address pointer by `sum`

Practice: Multiply integers

- Given the following C code to multiply two numbers using logic and bit movement operations, implement the equivalent in Assembly

```
int multiply(int a, int b){
  int res = 0;

  while (b != 0){
    if ((b & 1) != 0)
      res = res + a;
    a = a << 1;
    b = b >> 1;
  }

  return res;
}
```